Class-Based Components in React

An Alternative Way Of Building Components

A beginner-friendly guide — written in plain English with examples

1. What are Class Components — and Why Use Them?

A **Class Component** is a way of writing a React component using a JavaScript **class** instead of a plain function. Before React 16.8 (when Hooks were introduced), class components were the **only way** to use state and lifecycle features.

■ Reasons to know them:

- Many older codebases use them
- You may inherit legacy React projects
- Error Boundaries still require classes
- Helps you understand React history
- Some libraries were built with classes

■ Why most new code uses functions:

- Functional + Hooks = shorter code
- Easier to read and test
- Less boilerplate (no 'this')
- React team recommends functions
- All new features target Hooks

2. Adding Your First Class-Based Component

To create a class component, you **extend React.Component** and add a **render()** method that returns JSX. Think of it like writing a blueprint (class) instead of a recipe (function).

Side-by-side comparison:

Functional Component

```
function Hello({ name }) {
return (
<h1>Hello, {name}!</h1>
);
}
```

Class Component

```
class Hello extends
React.Component {
render() {
return (
<h1>Hello,
{this.props.name}!</h1>
);
}
}
```

> ***Key difference:*** *In class components, you always access props using* ***this.props*** *and you must have a* ***render()*** *method — it's what React calls to get your UI.*

3. Working with State & Events

In class components, state lives in **this.state** (an object set in the constructor). You update it with **this.setState()** — never directly. Events work the same as in functions, but methods need to be **bound to 'this'**.

Complete example — a counter:

```
import React from 'react';
class Counter extends React.Component {
constructor(props) {
super(props); // Always call super(props) first!
this.state = { count: 0 }; // Initialize state here
// Bind the method so 'this' works inside it
this.handleClick = this.handleClick.bind(this);
}
handleClick() {
this.setState({ count: this.state.count + 1 });
}
render() {
return (
Count: {this.state.count}
Add 1
);
}
}
```

Rule	Explanation
Always call super(props)	Required before using 'this' in the constructor
Never modify this.state directly	this.state.count = 5 is WRONG — use setState()
setState() merges, not replaces	Only the keys you pass get updated, rest stay
Use updater function for safety	setState(prev => ({ count: prev.count + 1 }))

4. The Component Lifecycle (Class-Based Only!)

Every class component goes through a **lifecycle** — it is born (mounted), lives and updates, then dies (unmounts). React provides special **lifecycle methods** you can override to run code at each stage. Functional components replicate this with **useEffect** instead.

MOUNTING	UPDATING	UNMOUNTING
`constructor()` ↓ `render()` ↓ `componentDidMount()`	`setState()` / new props ↓ `render()` ↓ `componentDidUpdate()`	`componentWillUnmount()` ↓ Component removed from screen

5. Lifecycle Methods In Action

Method	When it runs	Common use
`constructor()`	Before first render	Initialize state, bind methods
`componentDidMount()`	Right after first render	Fetch data, set up timers/subscriptions
`componentDidUpdate()`	After every re-render	React to prop/state changes, re-fetch data
`componentWillUnmount()`	Just before removed	Clear timers, cancel subscriptions
`shouldComponentUpdate()`	Before re-render	Return false to skip re-render (optimization)

Full example using multiple lifecycle methods:

```
class DataFetcher extends React.Component {
constructor(props) {
super(props);
this.state = { data: null, loading: true };
}
componentDidMount() {
// Runs once after component appears on screen
fetch('/api/data')
.then(res => res.json())
.then(data => this.setState({ data, loading: false }));
}
componentDidUpdate(prevProps) {
// Runs after every update — always compare to avoid infinite loop!
if (prevProps.userId !== this.props.userId) {
this.fetchUserData(this.props.userId);
}
}
componentWillUnmount() {
// Clean up before component disappears
clearInterval(this.timer);
}
```

```
render() {
if (this.state.loading) return Loading...;
return {this.state.data};
}
}
```

> ***Important:*** *Always check* ***prevProps*** *or* ***prevState*** *inside componentDidUpdate before calling setState — otherwise you'll create an infinite loop!*

6. Class-Based Components & Context

Context lets you share data (like theme, language, or logged-in user) across many components without passing props manually at every level. Class components can consume context using **static contextType** or the **Context.Consumer** wrapper.

Step 1 — Create context (same as functional components):

```
// ThemeContext.js
import React from 'react';
export const ThemeContext = React.createContext('light'); // default value
```

Step 2 — Consume in a class component (recommended way):

```
import { ThemeContext } from './ThemeContext';
class ThemedButton extends React.Component {
// Attach context to this class
static contextType = ThemeContext;
render() {
const theme = this.context; // Access context value via this.context
return (
I am a {theme} themed button!
);
}
}
```

Limitation: *A class component can only consume* ***one context*** *via contextType. If you need multiple contexts, use the older <Context.Consumer> wrapper approach — or switch to a functional component with multiple useContext() calls.*

7. Class-Based vs Functional Components: A Summary

Feature	Class Component	Functional Component
Syntax	class MyComp extends React.Component	function MyComp() { ... }
State	this.state + this.setState()	useState() Hook
Lifecycle	componentDidMount, etc.	useEffect() Hook
Context	static contextType (one only)	useContext() (multiple OK)
'this' keyword	Required everywhere	Not needed
Code length	More boilerplate	Shorter, cleaner
Error Boundary	Yes — supported	Not supported (use class)

Recommended?	**Legacy / special cases only**	**Yes — for new projects**

> ***Bottom line:*** *Use* ***functional components*** *for everything new. Learn class components so you can read and maintain older React code — and to write* ***Error Boundaries****, which still require classes.*

8. Introducing Error Boundaries

An **Error Boundary** is a special class component that **catches JavaScript errors** in its child component tree and shows a **fallback UI** instead of crashing the whole app. It's like a try-catch block, but for React components.

Without Error Boundary vs With Error Boundary:

■ Without Error Boundary	■ With Error Boundary
One broken component crashes the entire app.	Error is caught by the boundary.
Users see a blank white screen with no explanation.	A friendly fallback message is shown instead.
Very bad user experience!	Rest of the app keeps working!

How to create an Error Boundary:

```
class ErrorBoundary extends React.Component {
constructor(props) {
super(props);
this.state = { hasError: false };
}
// Called when a child throws an error
static getDerivedStateFromError(error) {
return { hasError: true }; // Switch to fallback UI
}
// Called for logging the error details
componentDidCatch(error, info) {
console.error('Error caught:', error, info);
// You could send this to an error tracking service like Sentry
}
render() {
if (this.state.hasError) {
return Something went wrong. Please try again.;
}
return this.props.children; // Render children normally if no error
}
```

```
}
```

How to use it — wrap any component tree:

```
function App() {
return (
{/* If this crashes, ErrorBoundary catches it */}
);
}
```

Error Boundaries DO catch	Error Boundaries do NOT catch
Errors in render() method	Errors in event handlers (use try-catch)
Errors in lifecycle methods	Errors in async code (setTimeout, fetch)
Errors in constructor of children	Errors in the boundary itself
Errors deep in the component tree	Errors during server-side rendering

> ***Pro tip:*** *Create multiple Error Boundaries at different levels of your app to isolate failures. A broken sidebar shouldn't crash your header or main content!*

Quick Reference — Class Component Cheat Sheet

Concept	Class Component Syntax
Initial state	`this.state = { key: value } in constructor`
Update state	`this.setState({ key: newValue })`
Access props	`this.props.propName`
Access state	`this.state.stateName`
After first render	`componentDidMount()`
After updates	`componentDidUpdate(prevProps, prevState)`
Before unmount	`componentWillUnmount()`
Consume context	`static contextType = MyContext; then this.context`
Catch child errors	`getDerivedStateFromError() + componentDidCatch()`